

Universidad Tecnológica de la Mixteca

Analizador Sintáctico

Implementación del algoritmo de Primeros y Siguientes

CompiladorEq3 — Asignación 3

Curso:	Compiladores
Profesor:	David Martínez Torres
Programa:	Ingeniería en Computación, quinto semestre

Equipo 3

Arenas Rojas Ivri Nitzani
Bautista Jiménez Israel
Hernández Torija Itzel Amairani
Juárez Jiménez Yarelli Sinai
López López Xochitl Nerina
Quiroz Martínez Maybelin

Oaxaca, México

Índice

Introducción **2**

1. Análisis **2**
1.1. El analizador sintáctico y las gramáticas libres de contexto . . . 2
1.2. Los conjuntos PRIMERO y SIGUIENTE . . . 2
1.3. Herramientas utilizadas . . . 3
1.4. Formato del archivo de la gramática . . . 3

2. Algoritmos de PRIMERO y SIGUIENTE **4**
2.1. Cálculo de PRIMERO . . . 4
2.2. Cálculo de SIGUIENTE . . . 4
2.3. Cálculo iterativo hasta un punto fijo . . . 4
2.4. Ejemplo paso a paso: Gramática 1 . . . 4

3. Descripción de la implementación **5**

4. Implementación en Python **6**
4.1. Arquitectura por capas y paquetes . . . 6
4.2. Representación de la gramática . . . 7
4.3. Cálculo de PRIMERO . . . 7
4.4. Cálculo de SIGUIENTE . . . 8
4.5. Manejo de errores en la gramática . . . 8
4.6. Relación con el analizador léxico . . . 9

5. Integración con la interfaz gráfica **10**

6. Diagrama de clases **13**

7. Pruebas **13**
7.1. Prueba 1: Gramática 1 (expresiones aritméticas) . . . 15
7.2. Prueba 2: Gramática 2 ($D \to D\ D$) . . . 16
7.3. Prueba 3: Gramática 3 (declaraciones con $T \to \varepsilon$) . . . 17
7.4. Prueba 4: Gramática 4 (sentencias) . . . 17
7.5. Prueba 5: Gramática 5 (arreglos) . . . 18
7.6. Prueba 6: Gramática 6 (expresiones LL(1)) . . . 19

Distribución de responsabilidades **20**

Conclusiones **21**

Recursos **22**

Referencias **22**

Introducción

En las dos primeras entregas el equipo construyó el analizador léxico del compilador de C++ a Java. Primero se hizo como un Autómata Finito Determinista programado a mano en Python (Asignación 1) y después generado automáticamente con Flex (Asignación 2). Este informe documenta la tercera entrega (Asignación 3), con la que el proyecto entra a la segunda fase del compilador: el **análisis sintáctico**, que corresponde al Tema 3 del curso.

Antes de construir un analizador sintáctico hay que conocer dos conjuntos de cada no terminal de la gramática. **PRIMERO**(A) es el conjunto de terminales con los que puede empezar una cadena derivada de A. **SIGUIENTE**(A) es el conjunto de terminales que pueden aparecer justo después de A en alguna forma sentencial. Con ellos se llenan la tabla de un analizador predictivo LL(1) y la tabla de reducciones de un analizador SLR, por lo que su cálculo es la base de las siguientes entregas.

El objetivo de la asignación es implementar el algoritmo que calcula ambos conjuntos para cualquier gramática libre de contexto leída de un archivo. El programa recibe un archivo de texto con los no terminales, los terminales, el símbolo inicial y las reglas de producción, y devuelve los conjuntos PRIMEROS y SIGUIENTES de cada no terminal. El cálculo se integró en la misma interfaz gráfica de las entregas anteriores, como una ventana hija modal que se abre desde el menú *Analizadores* → *Analizador sintáctico*.

El trabajo se organizó en siete partes: (1) análisis de los conceptos y del formato de entrada; (2) estudio de los algoritmos de PRIMERO y SIGUIENTE; (3) descripción de cómo se implementaron; (4) implementación en Python con la arquitectura por capas; (5) integración con la interfaz gráfica; (6) pruebas con las seis gramáticas de la asignación; y (7) distribución de responsabilidades entre los seis integrantes del equipo.

1 Análisis

1.1 El analizador sintáctico y las gramáticas libres de contexto

El analizador sintáctico recibe la tira de tokens que produce el analizador léxico y comprueba que siga la estructura del lenguaje. Esa estructura se describe con una **gramática libre de contexto**, que es una cuádrupla $G = (N, T, P, S)$ [5], donde:

- N es el conjunto de **no terminales** (variables), como E, T o F;
- T es el conjunto de **terminales** (los tokens), como `id`, `+` o `(`;
- P es el conjunto de **reglas de producción** de la forma $A \to \alpha$, con $A \in N$ y α una cadena de terminales y no terminales, posiblemente vacía (ε);
- $S \in N$ es el **símbolo inicial**.

1.2 Los conjuntos PRIMERO y SIGUIENTE

Según Aho *et al.* [1]:

- PRIMERO(α) es el conjunto de terminales con los que empiezan las cadenas que se derivan de α. Si $\alpha \Rightarrow^* \varepsilon$, también contiene ε.

- SIGUIENTE(A) es el conjunto de terminales a que pueden aparecer inmediatamente a la derecha de A en alguna forma sentencial, es decir, que existe una derivación $S \Rightarrow^* \alpha A a \beta$. Si A puede quedar al final de la cadena, también contiene el marcador de fin \$.

Por ejemplo, en la gramática de expresiones aritméticas (Gramática 1), toda expresión empieza con `(` o con `id`, así que PRIMERO(E) = { (id }. Después de una E completa solo pueden venir `+`, `)` o el fin de la cadena, así que SIGUIENTE(E) = { \$ +) }.

1.3 Herramientas utilizadas

Tabla 1: Herramientas utilizadas en la Asignación 3.

Herramienta	Uso
Python 3	Lenguaje del lector de gramáticas y del cálculo de los conjuntos.
Tkinter	Interfaz gráfica: ventana hija modal, diálogos para abrir y guardar archivos y mensajes de error.
PyInstaller	Empaqueta la interfaz como `Compilador.exe` para Windows, con las gramáticas de prueba incluidas.
Visual Studio Code	Edición del código y de las gramáticas de prueba.

1.4 Formato del archivo de la gramática

El enunciado pide que el archivo indique los no terminales, los terminales, el símbolo inicial y las reglas de producción [6]. El equipo definió el formato siguiente, que respeta el ejemplo de la asignación:

```
E T F            <- línea 1: no terminales
+ * ( ) id       <- línea 2: terminales
E                <- línea 3 (opcional): símbolo inicial
E→E+T             <- después, una regla de producción por línea
E→T
...
```

- La flecha puede escribirse →, `->` o `::=`.
- La cadena vacía se escribe ε, `eps` o dejando vacío el lado derecho.
- Se permiten alternativas con `|`, como en `F→(E) | id`, siempre que `|` no esté declarado como terminal.
- Los símbolos pueden ir juntos (`E→E+T`) o separados por espacios (`D → T id L ;`).
- Si se omite la línea 3, el símbolo inicial es el primer no terminal.

2 Algoritmos de PRIMERO y SIGUIENTE

Las reglas de ambos algoritmos se tomaron del enunciado de la asignación [6] y de la presentación del Tema 3 [7], y coinciden con las de Aho *et al.* [1, sección 4.4.2].

2.1 Cálculo de PRIMERO

Para calcular PRIMERO(X) de todo símbolo gramatical X se aplican las reglas siguientes hasta que no se pueda añadir ningún terminal ni ε a ningún conjunto:

1. Si X es terminal, PRIMERO(X) = $\{X\}$.
2. Si $X \to \varepsilon$ es una producción, se añade ε a PRIMERO(X).
3. Si $X \to Y_1 Y_2 \cdots Y_k$ es una producción, se añade PRIMERO(Y_1) $- \{\varepsilon\}$. Si $Y_1 \Rightarrow^* \varepsilon$, se añade también PRIMERO(Y_2) $- \{\varepsilon\}$, y así sucesivamente. Si todos los Y_j derivan ε, se añade ε.

2.2 Cálculo de SIGUIENTE

Para calcular SIGUIENTE(A) de todo no terminal A se aplican las reglas siguientes hasta que no se pueda añadir nada a ningún conjunto:

1. Se coloca \$ en SIGUIENTE(S), donde S es el símbolo inicial.
2. Si existe una producción $B \to \alpha A \beta$, todo PRIMERO(β) $- \{\varepsilon\}$ se añade a SIGUIENTE(A).
3. Si existe una producción $B \to \alpha A$, o $B \to \alpha A \beta$ con $\varepsilon \in$ PRIMERO(β), todo SIGUIENTE(B) se añade a SIGUIENTE(A).

2.3 Cálculo iterativo hasta un punto fijo

Las reglas se refieren unas a otras. Por ejemplo, con $E \to E + T$, PRIMERO(E) depende de sí mismo. Si se programaran con recursión directa, una gramática con recursividad por la izquierda provocaría un ciclo infinito. Por eso el equipo siguió el enfoque *iterativo de punto fijo* que presentan Louden [2] y Appel [3]: todos los conjuntos empiezan vacíos, se recorren todas las producciones aplicando las reglas, y el recorrido se repite mientras algún conjunto haya cambiado.

El algoritmo siempre termina, porque los conjuntos solo crecen y cada uno tiene como máximo $|T| + 1$ elementos (los terminales más ε o \$). Tarde o temprano una pasada completa no agrega nada, y en ese momento los conjuntos están completos.

2.4 Ejemplo paso a paso: Gramática 1

La Tabla 2 muestra el valor de los conjuntos al terminar cada pasada del ciclo para la Gramática 1. Los datos salen de ejecutar el programa. En PRIMERO la información fluye de abajo hacia arriba ($F \to T \to E$): se necesitan tres pasadas con cambios y una cuarta que confirma que ya no hay más. En SIGUIENTE, el) que $F \to (E)$ aporta a SIGUIENTE(E) llega a T y a F en la segunda pasada, por la regla 3 aplicada a $E \to T$ y a $T \to F$.

Tabla 2: Conjuntos al terminar cada pasada del ciclo (Gramática 1).

Pasada	E	T	F
PRIMERO			
1	$\emptyset$	$\emptyset$	{ (id }
2	$\emptyset$	{ (id }	{ (id }
3	{ (id }	{ (id }	{ (id }
4	sin cambios: termina		
SIGUIENTE			
1	{ $ +) }	{ $ + * }	{ $ + * }
2	{ $ +) }	{ $ +) * }	{ $ +) * }
3	sin cambios: termina		

3 Descripción de la implementación

El análisis de una gramática pasa por cuatro fases, cada una a cargo de una clase o función del paquete `persistencia`.

1. Lectura y validación (`LectorGramatica`). El texto de la gramática se divide en el encabezado (no terminales, terminales y símbolo inicial) y las reglas. Una línea se reconoce como regla si tiene un único símbolo antes de la flecha, así que la línea de terminales no se confunde con una regla aunque contenga `->`. Cada lado derecho se divide en símbolos por *coincidencia más larga* contra los símbolos declarados: así `E'→+TE'` se lee como `E'`, `+`, `T`, `E'`, y no como `E` seguido de un apóstrofo. Las alternativas con `|` se separan en producciones independientes. El resultado es un objeto `Gramatica`.

2. Cálculo de PRIMERO (`calcular_primeros()`). Se aplican las tres reglas de la sección 2.1 hasta el punto fijo. La regla 3 aplicada a una cadena cualquiera se concentra en la función `primeros_de()`, que también usa SIGUIENTE.

3. Cálculo de SIGUIENTE (`calcular_siguientes()`). Se ejecuta después de PRIMERO, porque lo necesita, y aplica las tres reglas de la sección 2.2 hasta el punto fijo.

4. Orden y formato de la salida (`reporte()`). Un conjunto no tiene orden, pero el ejemplo de la asignación muestra sus elementos en un orden concreto (`S(T)={ $ + ) * }`). Para que la salida se pueda comparar directamente con el ejemplo y con el cálculo manual, cada conjunto se presenta en el orden en que sus elementos se encontrarían al aplicar las reglas a mano. Después se le da el formato `P(A)={ ... }` y `S(A)={ ... }`.

4 Implementación en Python

4.1 Arquitectura por capas y paquetes

El proyecto sigue el *Ejemplo de la Arquitectura de Software y uso de GUI en Eclipse* del profesor [8], con los paquetes de Java convertidos a paquetes de Python. La capa de **persistencia** contiene las clases que leen la gramática y calculan los conjuntos, y la capa de **presentación** contiene las ventanas. Los paquetes nuevos de esta entrega se agregaron junto a los del analizador léxico, sin modificarlos.

Igual que en Eclipse, cada paquete se nombra con puntos. En Python, cada punto corresponde a una carpeta dentro de `src` que contiene un archivo `__init__.py`; por ejemplo, el paquete `compi.presentacion.asintactico` es la carpeta `src/compi/presentacion/asintactico/`. La Tabla 3 muestra todos los paquetes del proyecto.

Tabla 3: Paquetes de `src` y su contenido. Los marcados con ⋆ son nuevos en la Asignación 3.

Paquete	Contenido
`compi.persistencia.analizadorLexico.alexico`	`afd_lexico.py`, `analizador_lexico.py` (Asignación 1)
`compi.persistencia.analizadorLexico.flex`	`compilador_flex.py` (Asignación 2)
`compi.persistencia.analizadorLexico.generales`	`tokens.py`, `lector_archivo.py`
`compi.persistencia.analizadorSintacticoLR.generales` ⋆	`gramatica.py`: `Gramatica`, `LectorGramatica`, `ErrorGramatica`
`compi.persistencia.analizadorSintacticoLR.primerosYsiguientes` ⋆	`primeros_siguientes.py`: `PrimerosSiguientes`
`compi.presentacion`	`compilador.py` (`main`) y `ventana_principal.py` (menús y logo)
`compi.presentacion.alexico`	Pantallas y subventanas del analizador léxico (AFD y Flex)
`compi.presentacion.asintactico` ⋆	`ventana_primeros_siguientes.py`: `VentanaPrimerosSiguientes`

Además de `src`, la carpeta del proyecto contiene los recursos que usan los analizadores:

```
Arquitectura_software_PyS/
|-- src/              paquetes compi.persistencia y compi.presentacion
|-- codigosFuentes/   programas .cpp de prueba del analizador léxico
|-- gramaticas/       gramáticas g1 ... g6 (.txt) de la Asignación 3
|-- flex/             lexer.l del analizador en Flex
|-- imagenes/         logo_utm.png
|-- herramientas/     win_flex y TinyCC (para compilar lexer.l en Windows)
|-- README.md         instrucciones de uso
‘-- iniciar.sh        inicia la interfaz en Linux y macOS
```

El proyecto se entrega empaquetado en el archivo `Arquitectura_software_PyS.zip`, con la estructura anterior. No incluye el ejecutable de Windows (`Compilador.exe`), que se entrega por separado, ni los archivos que se generan al usarlo: las carpetas `__pycache__` de Python y los archivos que Flex crea al compilar y ejecutar `lexer.l` (`lex.yy.c`, el ejecutable del analizador y los `.txt` de

resultados).

4.2 Representación de la gramática

La clase `Gramatica` guarda las cuatro componentes de G. Cada producción es una tupla `(A, (Y1, Y2, ..., Yk))`. La producción $A \to \varepsilon$ se guarda con el lado derecho vacío, `(A, ())`: así ε no es un símbolo más de la gramática, sino la ausencia de símbolos, que es justamente lo que significa.

```
class Gramatica:
    def __init__(self, no_terminales, terminales, inicial, producciones):
        self.no_terminales = list(no_terminales)
        self.terminales = list(terminales)
        self.inicial = inicial
        self.producciones = list(producciones)   # [(A, (Y1, ..., Yk)), ...]

    def es_no_terminal(self, simbolo):
        return simbolo in self.no_terminales
```

4.3 Cálculo de PRIMERO

La función `primeros_de()` implementa la regla 3 para cualquier cadena $Y_1 Y_2 \cdots Y_k$. Recorre los símbolos acumulando sus primeros sin ε y se detiene en el primero que no deriva ε. Si llega al final sin detenerse, agrega ε. Con la cadena vacía devuelve $\{\varepsilon\}$ directamente, así que la regla 2 queda incluida en la 3 y no necesita código propio.

```
def primeros_de(self, simbolos):
    """PRIMERO de una cadena Y1 Y2 ... Yk (regla 3)."""
    resultado = set()
    for y in simbolos:
        resultado |= self.primeros[y] - {EPSILON}
        if EPSILON not in self.primeros[y]:
            return resultado
    resultado.add(EPSILON)       # todos los Yj derivan ε (o cadena vacía)
    return resultado

def calcular_primeros(self):
    g = self.gramatica
    self.primeros = {t: {t} for t in g.terminales}        # regla 1
    for a in g.no_terminales:
        self.primeros[a] = set()
    cambio = True
    while cambio:                                         # punto fijo
        cambio = False
        for x, derecha in g.producciones:
            nuevos = self.primeros_de(derecha) - self.primeros[x]
            if nuevos:                                    # reglas 2 y 3
                self.primeros[x] |= nuevos
                cambio = True
```

Los conjuntos se guardan como `set` de Python, así que un terminal nunca se repite. El ciclo `while` se detiene en cuanto una pasada completa no agrega nada.

4.4 Cálculo de SIGUIENTE

Para cada producción $B \to Y_1 \cdots Y_k$ y cada posición i donde Y_i es un no terminal A, la cola $\beta = Y_{i+1} \cdots Y_k$ es `derecha[i+1:]`. Con un solo llamado a `primeros_de(`β`)` se resuelven las reglas 2 y 3. Si β es vacía, la función devuelve $\{\varepsilon\}$, y eso activa la regla 3 sin un caso especial.

```
def calcular_siguientes(self):
    g = self.gramatica
    self.siguientes = {a: set() for a in g.no_terminales}
    self.siguientes[g.inicial].add(FIN)                    # regla 1: $
    cambio = True
    while cambio:
        cambio = False
        for b, derecha in g.producciones:
            for i, a in enumerate(derecha):
                if not g.es_no_terminal(a):
                    continue
                primeros_beta = self.primeros_de(derecha[i + 1:])
                nuevos = primeros_beta - {EPSILON}         # regla 2
                if EPSILON in primeros_beta:               # regla 3
                    nuevos |= self.siguientes[b]
                nuevos -= self.siguientes[a]
                if nuevos:
                    self.siguientes[a] |= nuevos
                    cambio = True
```

Este diseño resuelve automáticamente el caso $D \to D\,D$ de la Gramática 2. La primera D recibe PRIMERO(D) por la regla 2, y la segunda recibe SIGUIENTE(D) por la regla 3.

4.5 Manejo de errores en la gramática

Si el archivo no respeta el formato, `LectorGramatica` lanza la excepción `ErrorGramatica` con un mensaje que indica la línea del problema. La ventana la atrapa y la muestra en un cuadro de diálogo, sin cerrar el programa (Figura 1).

Tabla 4: Errores que detecta el lector de gramáticas.

Situación	Mensaje
Archivo vacío	La gramática está vacía.
No hay ninguna línea con flecha	No se encontraron reglas de producción (A → α).
Encabezado con más de 3 líneas	Antes de las reglas se esperan 2 líneas y, opcionalmente, una tercera.
Un símbolo es terminal y no terminal a la vez	Estos símbolos están declarados como no terminales y como terminales a la vez.
El símbolo inicial no es un no terminal	Línea n: el símbolo inicial no está entre los no terminales.
Lado izquierdo no declarado	Línea n: «A» está del lado izquierdo pero no está declarado como no terminal.
Símbolo del lado derecho no declarado	Línea n: «x» no es un símbolo declarado.
No terminal sin reglas	Estos no terminales no tienen reglas de producción.

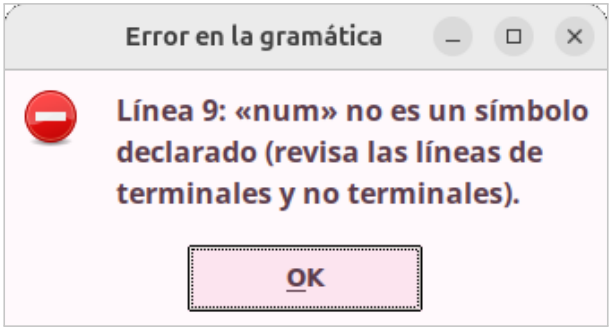


Figura 1: Error al analizar la Gramática 1 con una regla extra `F→num`: `num` no está declarado como terminal.

4.6 Relación con el analizador léxico

El cálculo de primeros y siguientes **no usa el analizador léxico** de las entregas anteriores. Ni `PrimerosSiguientes` ni `LectorGramatica` importan `AFDAlexico`, `AnalizadorLexico` ni el analizador en Flex. La razón es que la entrada de este algoritmo no es un programa en C++, sino una *gramática*, cuyos símbolos (`E`, `T'`, →) no son lexemas de C++. Los terminales `id` y `nint` aparecen como *nombres de tokens*, no como lexemas que haya que clasificar.

Sí se aprovecharon tres cosas de las entregas anteriores:

- **La coincidencia más larga.** El lector de la gramática divide `E'→+TE'` con el mismo principio que el analizador léxico usa para distinguir `<` de `<=`: prueba primero los símbolos declarados más largos.
- **La arquitectura por capas.** El módulo nuevo se agregó a los mismos paquetes `persistencia` y `presentacion`.
- **El estilo de la interfaz.** La ventana reutiliza los colores y fuentes de `estilos.py`, el módulo de estilos del analizador léxico, para que todo el compilador tenga la misma apariencia.

La conexión entre ambas fases llegará en las siguientes entregas. El analizador sintáctico recibirá como entrada la tira de tokens del analizador léxico, y usará los conjuntos calculados aquí para construir su tabla de análisis.

5 Integración con la interfaz gráfica

Al iniciar, la interfaz muestra el logo de la Universidad como pantalla principal (Figura 2). Al menú *Analizadores* se le agregó el submenú *Analizador sintáctico*, con la opción *Algoritmo Primeros y Siguientes*.

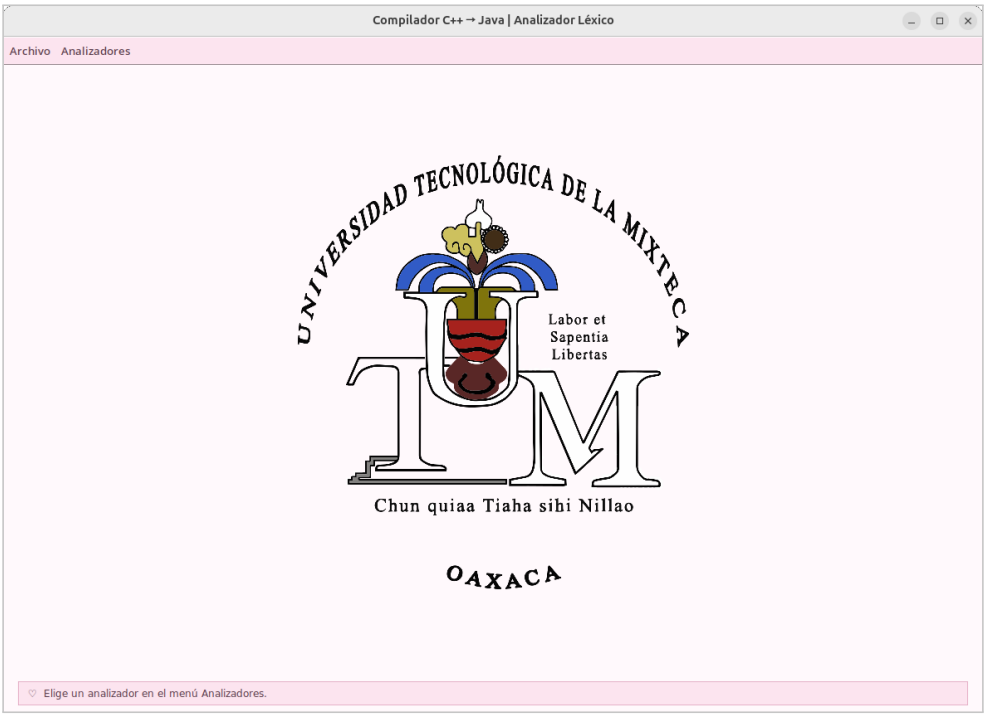


Figura 2: Pantalla principal del compilador. El cálculo se abre desde *Analizadores* → *Analizador sintáctico* → *Algoritmo Primeros y Siguientes*.

En el ejemplo de arquitectura del profesor, la ventana principal abre cada algoritmo como un `JDialog` modal (`new VentanaAThompson(this, true)`). En Python el equivalente es la clase `VentanaPrimerosSiguientes`, que hereda de `tk.Toplevel` [9]:

```
# VentanaPrincipal
def primeros_siguientes(self):
    VentanaPrimerosSiguientes(self, True)        # ventana hija modal

# VentanaPrimerosSiguientes(tk.Toplevel)
def __init__(self, padre, modal):
    super().__init__(padre.root)
    self.padre = padre
    self.lector = LectorGramatica()
    self.resultado = None
    self.ini()
    if modal:
        self.grab_set()      # la ventana principal no responde hasta cerrar esta
```

La ventana tiene tres zonas (Figura 3):

- **Formulario:** botón *Abrir* para elegir un archivo de la carpeta `gramaticas`, la ruta seleccionada y los botones *Analizar*, *Limpiar* y *Guardar resultados*.
- **Panel izquierdo:** la gramática, editable, con una ayuda sobre el formato del archivo.
- **Panel derecho:** el símbolo inicial, los no terminales, los terminales y los conjuntos PRIMEROS y SIGUIENTES. Debajo, una barra de estado con mensajes.

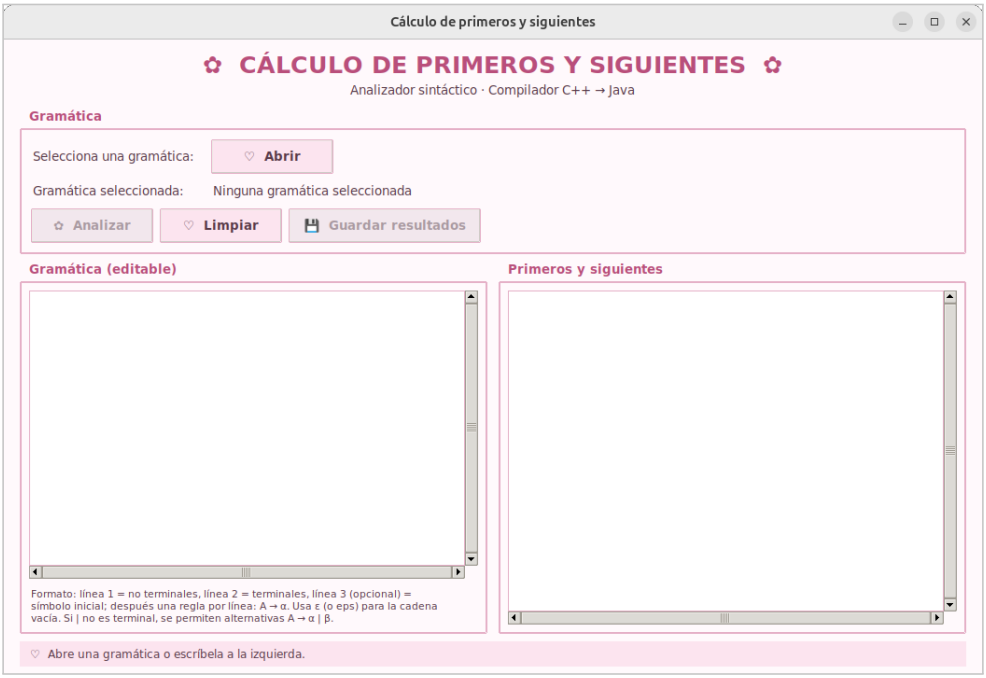


Figura 3: Ventana *Cálculo de primeros y siguientes* recién abierta. *Analizar* y *Guardar resultados* siguen desactivados porque todavía no hay gramática.

Los botones hacen lo siguiente:

- **Abrir** (`abrir_archivo()`): muestra un `filedialog` que empieza en la carpeta `gramaticas`, lee el archivo con `LectorGramatica.cargar_archivo()` y lo copia al panel izquierdo (Figura 4).

- **Analizar** (`analizar()`): toma el texto *actual* del panel, de modo que la gramática se puede corregir o escribir a mano sin crear un archivo. Lo pasa a `LectorGramatica.leer()`, calcula `PrimerosSiguientes(gramatica).calcular()` y escribe los resultados en el panel derecho.
- **Guardar resultados** (`guardar_resultados()`): escribe en un `.txt` la salida de `reporte()`, con el mismo formato que el ejemplo de la asignación.
- **Limpiar** (`limpiar()`): vacía ambos paneles.

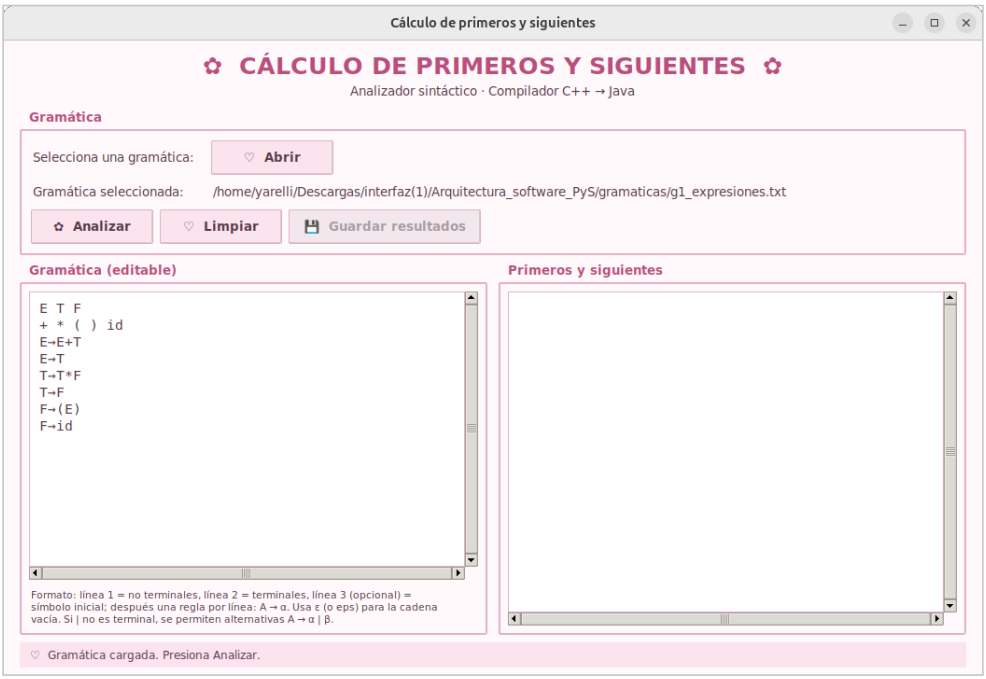


Figura 4: Gramática 1 (`g1_expresiones.txt`) cargada con el botón *Abrir*, antes de analizarla.

La ventana pertenece a la capa de presentación: solo lee lo que escribe el usuario y muestra resultados. Todo el cálculo está en la capa de persistencia, que no importa nada de `tkinter`. Por eso el algoritmo también se puede ejecutar sin interfaz, por ejemplo desde la terminal para hacer pruebas. Para iniciar la interfaz, desde la carpeta `src`:

```
$ python3 -m compi.presentacion.compilador
```

6 Diagrama de clases

Las clases nuevas de esta entrega se reparten en las dos capas de la arquitectura. La Figura 5 muestra sus atributos, sus métodos principales y cómo se relacionan.

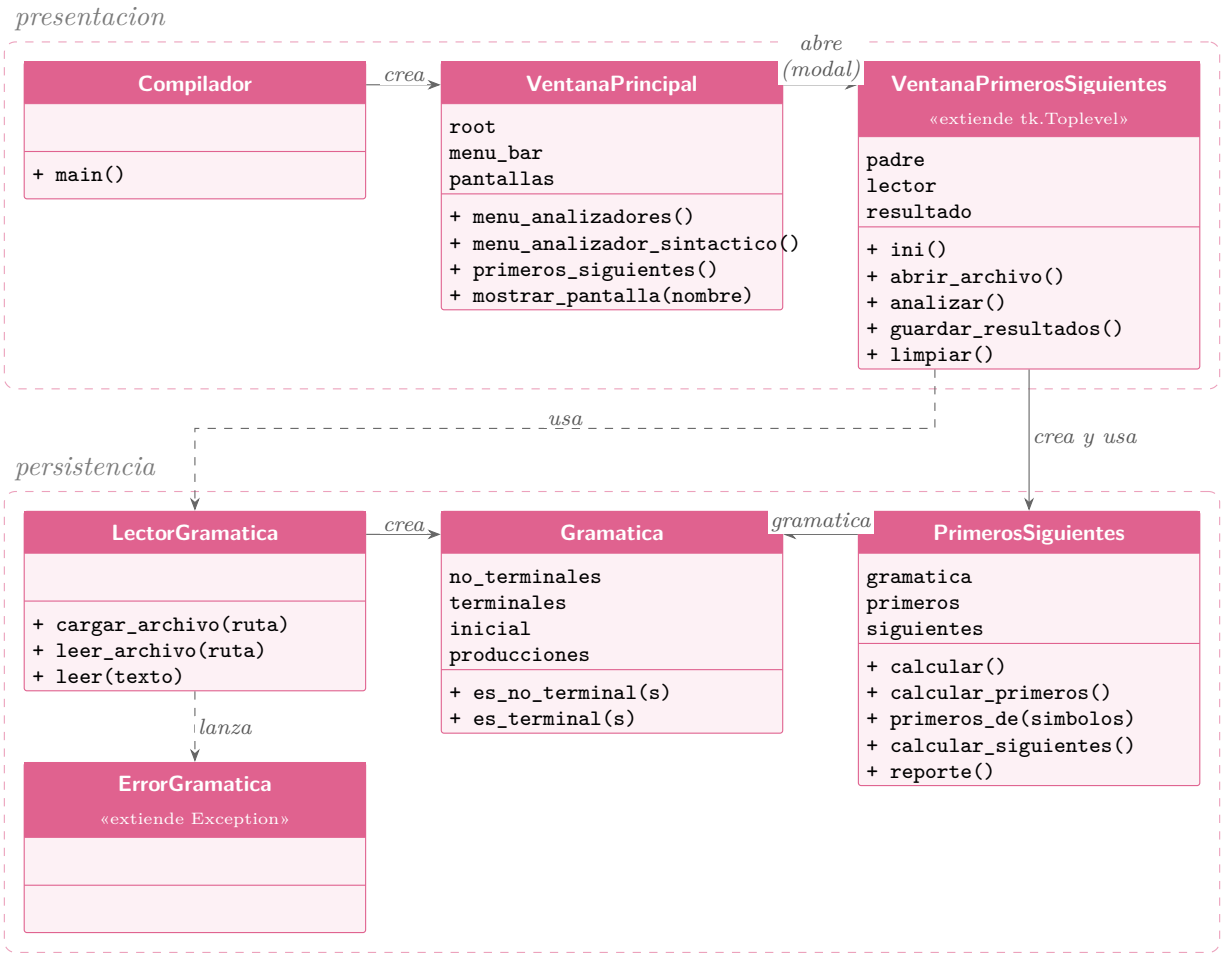


Figura 5: Diagrama de clases del cálculo de primeros y siguientes. Flecha continua: asociación o creación; flecha punteada: dependencia.

7 Pruebas

Las pruebas se ejecutaron desde la interfaz (*Abrir* → *Analizar*) con las seis gramáticas de la asignación, guardadas en la carpeta `gramaticas`. Los resultados se compararon contra el cálculo manual de cada gramática y, en el caso de la Gramática 1, contra la salida del ejemplo de la asignación. La Tabla 5 resume las pruebas, y las figuras siguientes muestran la salida real del programa.

Tabla 5: Gramáticas de prueba y lo que ejercita cada una.

Gram.	Archivo	No term.	Prod.	Caso que prueba
1	`g1_expresiones.txt`	3	6	Recursividad por la izquierda ($E \to E+T$); coincide con el ejemplo de la asignación.
2	`g2_declaraciones.txt`	3	6	No terminal repetido ($D \to D\,D$) y regla $L \to \varepsilon$.
3	`g3_declaraciones.txt`	3	7	Varios no terminales anulables en la misma regla (T y L).
4	`g4_sentencias.txt`	3	9	Recursividad por la izquierda en S, E y L; terminal de varios caracteres (`:=`).
5	`g5_arreglos.txt`	4	7	No terminal con apóstrofo (V') que deriva ε.
6	`g6_expresiones_ll1.txt`	5	8	Gramática LL(1) escrita con alternativas `|` y ε.

Tabla 6: Resultados del programa para las seis gramáticas.

Gram.	No term.	PRIMEROS	SIGUIENTES
1	E	{ (id }	{ $ +) }
	T	{ (id }	{ $ +) * }
	F	{ (id }	{ $ +) * }
2	D	{ float int }	{ $ float int }
	L	{ , ε }	{ ; }
	T	{ float int }	{ id }
3	D	{ float int id , ; }	{ $ }
	L	{ id , ε }	{ ; }
	T	{ float int ε }	{ id , ; }
4	S	{ id print }	{ $; , }
	E	{ id num (}	{ $; , +) }
	L	{ id num (}	{) , }
5	S	{ int char float }	{ $ }
	V	{ id }	{ ; }
	V'	{ [ε }	{ ; }
	T	{ int char float }	{ id }
6	E	{ (id }	{ $) }
	E'	{ + ε }	{ $) }
	T	{ (id }	{ + $) }
	T'	{ * ε }	{ + $) }
	F	{ (id }	{ * + $) }

7.1 Prueba 1: Gramática 1 (expresiones aritméticas)

Es la gramática del ejemplo de la asignación. La salida coincide exactamente con la del enunciado, incluso en el orden de los elementos de cada conjunto.

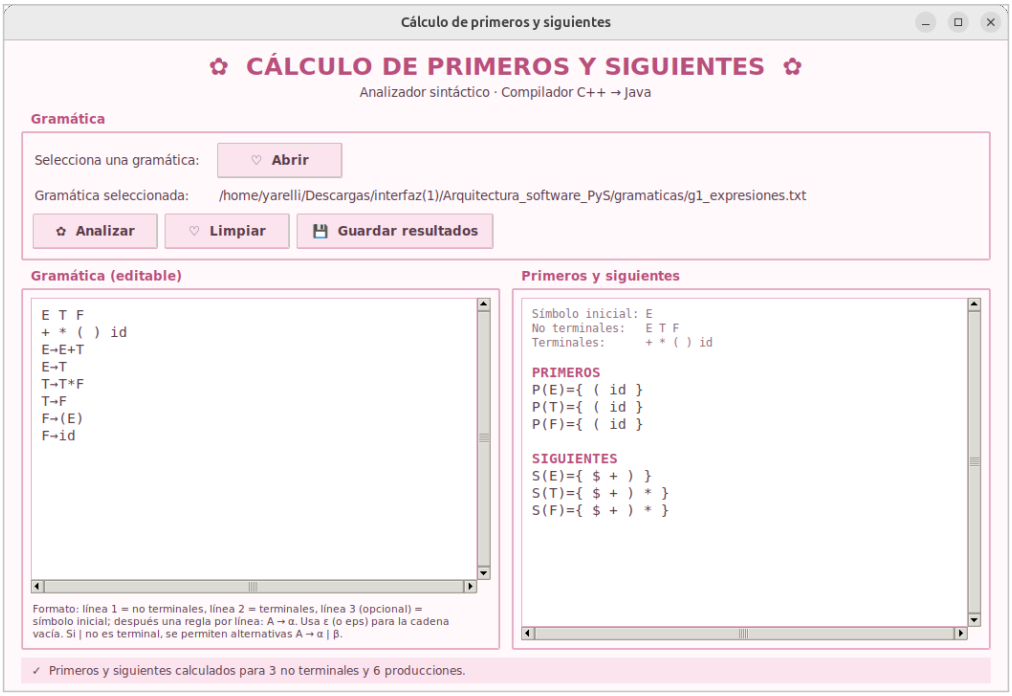


Figura 6: Resultado de la Gramática 1 (`g1_expresiones.txt`).

7.2 Prueba 2: Gramática 2 ($D \to D\ D$)

Como D aparece seguida de otra D, SIGUIENTE(D) recibe PRIMERO(D) = { float int }, además del $ por ser el símbolo inicial. L deriva ε, y por eso $\varepsilon \in$ PRIMERO(L).

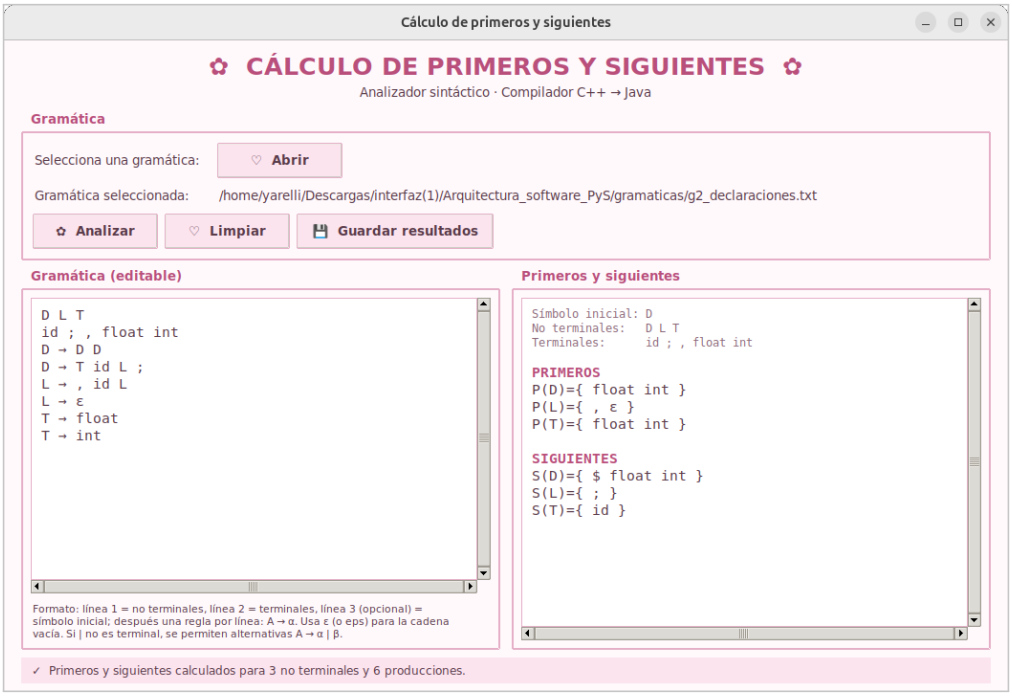


Figura 7: Resultado de la Gramática 2 (`g2_declaraciones.txt`).

7.3 Prueba 3: Gramática 3 (declaraciones con $T \to \varepsilon$)

En $D \to T\, L\, ;$, tanto T como L pueden ser vacías, así que PRIMERO(D) reúne los primeros de T, de L y el `;` final. Por la misma razón, SIGUIENTE(T) contiene PRIMERO(L) sin ε, más el `;`.

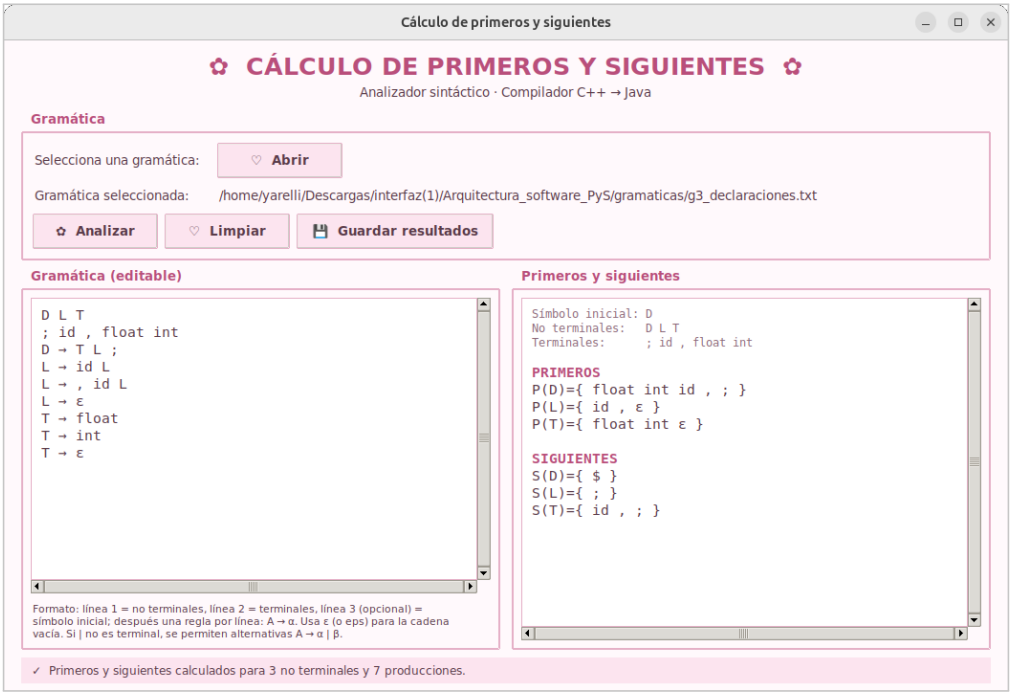


Figura 8: Resultado de la Gramática 3 (`g3_declaraciones.txt`).

7.4 Prueba 4: Gramática 4 (sentencias)

Tiene recursividad por la izquierda en los tres no terminales. El cálculo iterativo la resuelve sin caer en un ciclo infinito. El terminal `:=` tiene dos caracteres y se reconoce correctamente gracias a la coincidencia más larga del lector.

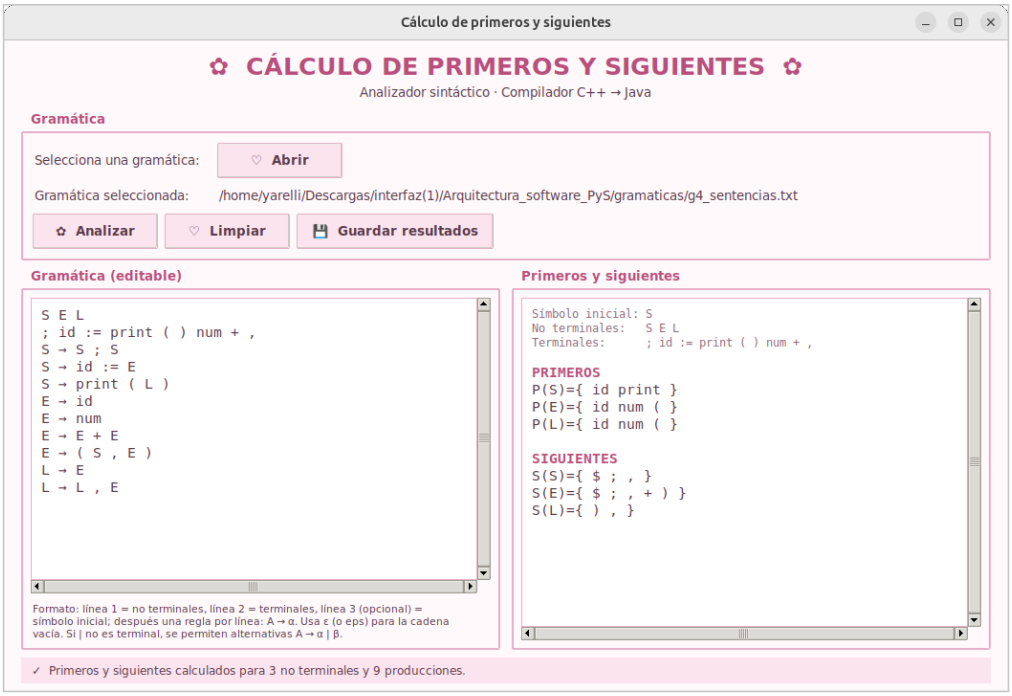


Figura 9: Resultado de la Gramática 4 (`g4_sentencias.txt`).

7.5 Prueba 5: Gramática 5 (arreglos)

V' aparece al final de $V \to \mathrm{id}\, V'$, así que SIGUIENTE(V') hereda SIGUIENTE(V) = { ; } por la regla 3. El apóstrofo se lee como parte del símbolo V', no como un terminal aparte.

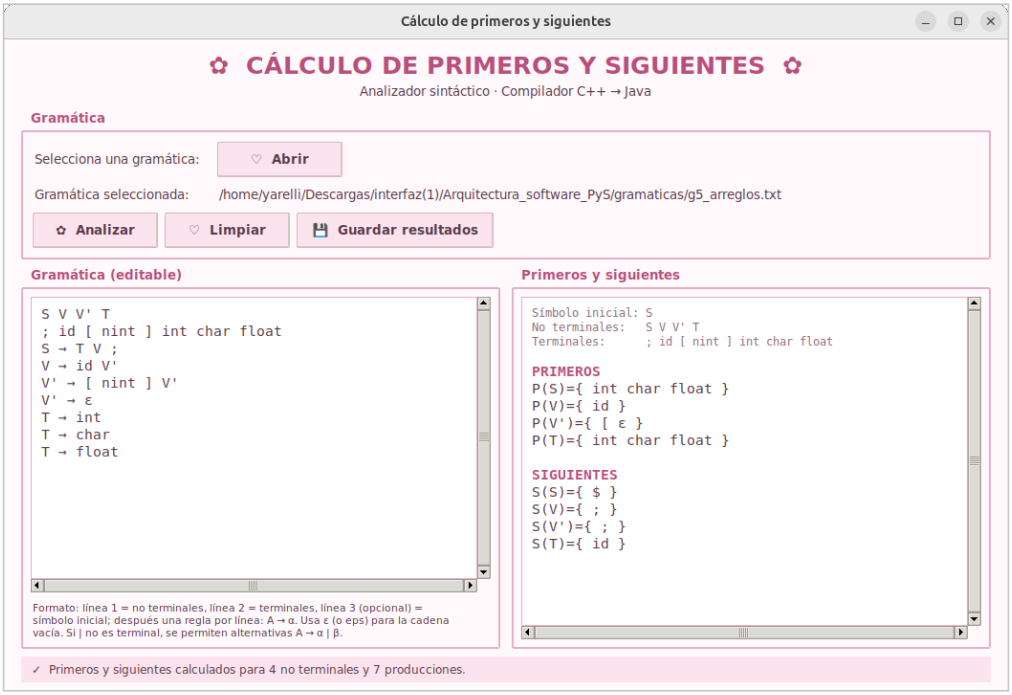


Figura 10: Resultado de la Gramática 5 (`g5_arreglos.txt`).

7.6 Prueba 6: Gramática 6 (expresiones LL(1))

Es la versión sin recursividad por la izquierda de la Gramática 1, escrita con alternativas `|`. El lector separa cada alternativa en una producción, por eso la ventana reporta 8 producciones aunque el archivo tenga 5 reglas.

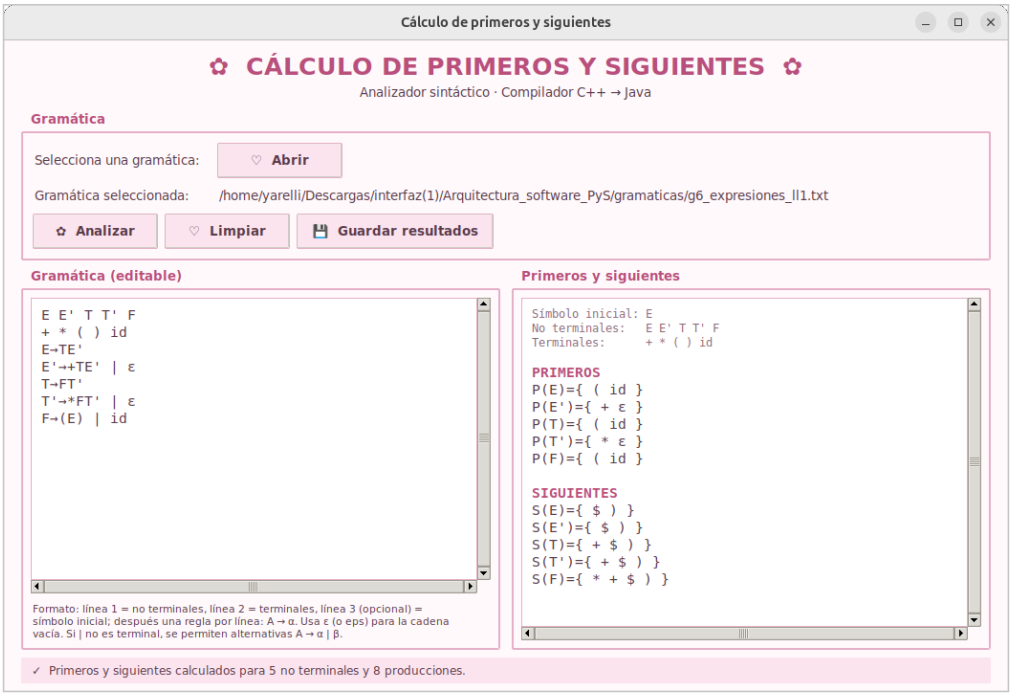


Figura 11: Resultado de la Gramática 6 (`g6_expresiones_ll1.txt`).

En las seis gramáticas los conjuntos calculados por el programa coincidieron con el cálculo manual.

Distribución de responsabilidades

Tabla 7: Módulos del cálculo de primeros y siguientes, funciones y responsables.

Módulo	Clase y funciones	Responsable	Actividad
Estructura y menú	`VentanaPrincipal`: `menu_analizador_sintactico()`, `primeros_siguientes()`; paquetes `persistencia` y `presentacion`	Arenas Rojas I. N.	Organiza el proyecto por capas, agrega el submenú *Analizador sintáctico* y abre la ventana hija modal. Coordina la planificación y el informe.
Interfaz gráfica	`VentanaPrimerosSiguientes`: `ini()`, `abrir_archivo()`, `analizar()`, `limpiar()`, `guardar_resultados()`	Bautista Jiménez I.	Construye la ventana con los botones, el panel de la gramática, el panel de resultados y la barra de estado.
Lector de gramática	`Gramatica`, `LectorGramatica`, `ErrorGramatica`	Hernández Torija I. A.	Lee el archivo, separa símbolos y alternativas, valida el formato y reporta errores con su línea. Elabora el diagrama de clases.
PRIMERO	`calcular_primeros()`, `primeros_de()`, `_orden_primeros()`	Juárez Jiménez Y. S.	Calcula PRIMERO de cada símbolo y de cadenas, repitiendo las reglas hasta que no haya cambios.
SIGUIENTE y salida	`calcular_siguientes()`, `_orden_siguientes()`, `formatear()`, `reporte()`	López López X. N.	Calcula SIGUIENTE de cada no terminal y da formato `P(A)={...}` y `S(A)={...}`.
Pruebas	`gramaticas/g1` a `g6`; `Compilador.exe`	Quiroz Martínez M.	Prepara las seis gramáticas, ejecuta las pruebas, las compara con el cálculo manual y prepara el ejecutable de Windows.

Tabla 8: Cronograma de la Asignación 3 (29 de septiembre al 7 de octubre de 2026).

Fecha	Fase	Actividades	Responsables
29/09	Análisis y reparto	Requerimientos, estudio de los algoritmos de PRIMERO y SIGUIENTE y reparto de módulos.	Todo el equipo
30/09	Diseño	Paquetes por capas, formato del archivo de gramática, clases `Gramatica` y `PrimerosSiguientes`, gramáticas de prueba.	Arenas, Hernández, Juárez, Quiroz
01/10 al 02/10	Implementación	Menú, ventana hija, lector de gramática, cálculo de PRIMERO y de SIGUIENTE.	Arenas, Bautista, Hernández, Juárez, López
05/10	Integración	Orden y formato de la salida, conexión de la ventana con el lector y el cálculo.	Todo el equipo
06/10	Pruebas e informe	Pruebas con las seis gramáticas, diagrama de clases e informe.	Quiroz, Hernández, Arenas, Juárez, López
07/10	Revisión y entrega	Reunión final, ensamblado del informe y entrega.	Todo el equipo

Conclusiones

El algoritmo de primeros y siguientes se implementó tal como lo plantean el enunciado y la bibliografía del curso: las tres reglas de cada conjunto se aplican sobre todas las producciones hasta alcanzar un punto fijo. Con este enfoque iterativo, el mismo código resuelve sin casos especiales las gramáticas con recursividad por la izquierda ($E \to E+T$, $S \to S;S$), las que tienen un no terminal repetido ($D \to D\,D$), las que tienen varios no terminales anulables en una regla y las que usan no terminales con apóstrofo (V', E', T'). Los resultados de las seis gramáticas coincidieron con el ejemplo de la asignación y con el cálculo manual.

Dos decisiones de diseño simplificaron mucho el código. La primera fue representar ε como un lado derecho vacío en lugar de un símbolo más. La segunda fue concentrar la regla 3 de PRIMERO en una sola función, `primeros_de()`. Con ella, la regla 2 de PRIMERO y las reglas 2 y 3 de SIGUIENTE se reducen a una sola llamada sobre la cola β de cada producción.

Mantener la arquitectura por capas permitió agregar el analizador sintáctico sin modificar el léxico. El cálculo vive en la capa de persistencia y no depende de la interfaz, y la nueva ventana solo se conectó al menú existente. Con los conjuntos PRIMERO y SIGUIENTE ya calculados, el siguiente paso es construir la tabla de un analizador sintáctico LL(1) o SLR que reciba la tira de tokens del analizador léxico.

Recursos

Enlace de Google Drive con las actividades realizadas por el equipo: *informeActividadesEq3*.

Referencias

[1] Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2008). *Compiladores: principios, técnicas y herramientas* (2.ª ed.). Pearson Educación.

[2] Louden, K. C. (2004). *Construcción de compiladores: principios y práctica*. Thomson.

[3] Appel, A. W. (2002). *Modern Compiler Implementation in Java* (2nd ed.). Cambridge University Press.

[4] Cooper, K. D., & Torczon, L. (2012). *Engineering a Compiler* (2nd ed.). Morgan Kaufmann.

[5] Hopcroft, J. E., Motwani, R., & Ullman, J. D. (2007). *Introducción a la teoría de autómatas, lenguajes y computación* (3.ª ed.). Pearson Educación.

[6] Martínez Torres, D. (2026). *Asignación 3: Implementación del algoritmo de Primeros y Siguientes*. Curso de Compiladores, Instituto de Computación, Universidad Tecnológica de la Mixteca.

[7] Martínez Torres, D. *Tema 3. Análisis sintáctico* (presentación del curso). Universidad Tecnológica de la Mixteca.

[8] Martínez Torres, D. *Ejemplo de la Arquitectura de Software y uso de GUI en Eclipse*. Material del curso de Compiladores, Universidad Tecnológica de la Mixteca.

[9] Python Software Foundation. *tkinter — Python interface to Tcl/Tk*. Documentación de Python 3. `https://docs.python.org/3/library/tkinter.html`